

WOW64 兼容性检测与 32/64 位 PEB 差异

● 为什么“位数”是个问题

Windows 系统有两套完全不同的内存布局逻辑：

- 64 位进程：原生运行在 64 位地址空间，PEB 在 gs:[0x60]，指针是 8 字节。
- 32 位进程：在 64 位 Windows 上通过 WOW64（Windows32-bit on Windows64-bit）子系统运行，PEB 在 fs:[0x30]，指针是 4 字节。

关键陷阱：如果用 64 位的偏移量（比如 PEB+0x18 找 Ldr）去读 32 位进程的 PEB，会读到一堆错位的数据，轻则输出乱码，重则直接崩溃。

所以，在跨进程读取 PEB、遍历模块、注入 Shellcode 之前，必须先判断目标进程的位数。

● WOW64 机制的物理本质

建立画面：

- 一个 32 位程序 mushiai_chs.exe 运行在 64 位 Windows 上。
- 系统为它在 C:\Windows\SysWOW64 里准备了 32 位的 ntdll.dll、kernel32.dll。
- 它的 fs:[0x30] 指向 32 位 PEB。
- 但同时，系统还会在 64 位地址空间里，为这个 WOW64 进程维护一个 64 位 PEB（用来跟内核沟通）。

● 两个辅助 API（格外补充）

GetModuleHandleA API 介绍

作用：给一个模块名（比如“ntdll.dll”），返回这个模块当前加载在内存里的基址。

```
HMODULE hNtdll=GetModuleHandleA("ntdll.dll");  
//如果成功，hNtdll 就是 ntdll.dll 的基址  
//如果失败，返回 NULL
```

物理类比：你走进图书馆，问前台：“XX 书在哪个架子上？”前台告诉你一个书架编号。

GetModuleHandleA 就是这个“前台”。

与 EnumProcessModules 对比：EnumProcessModules 是“把图书馆所有书都列出来”，而 GetModuleHandleA 是“直接问某一本书在哪”。

GetProcAddress API 介绍

作用：给一个模块基址和一个函数名，返回这个函数的内存地址。

```
//假设 hNtdll 是 ntdll.dll 的基址  
pNtQueryInformationProcess NtQuery=(pNtQueryInformationProcess)  
GetProcAddress(hNtdll,"NtQueryInformationProcess");  
//如果成功，NtQuery 就是 NtQueryInformationProcess 函数的地址
```

物理类比：拿着书架编号（hNtdll），问前台：“《百科全书》第 132 页的

NtQueryInformationProcess 词条在哪？”前台给你一个页码。

● 利用链路

背景：你想调用 NtQueryInformationProcess 这个函数，来检测目标进程是不是 32 位。

问题：这个函数不是标准 C++ 库函数，不在 <iostream> 里，也不在 kernel32.lib 里。它藏在 ntdll.dll 里，而且系统没有默认把它链接进来。直接写 NtQueryInformationProcess(...) 编译器会报错：“未定义”。

解决办法：分三步走。

- 第一步：拿到 ntdll.dll 这个 DLL 在内存里的基址。

```
HMODULE hNtdll=GetModuleHandleA("ntdll.dll");
```

- 第二步：在这个 DLL 的“导出表”里，找到 NtQueryInformationProcess 函数的地址。

```
pNtQueryInformationProcess NtQuery=(pNtQueryInformationProcess)
GetProcAddress(hNtdll, "NtQueryInformationProcess");
```

- 第三步：用这个地址调用函数。

```
NtQuery(hProcess, ProcessWow64Information, &wow64Info, sizeof(wow64Info), NULL);
```

完整利用链：

```
GetModuleHandleA("ntdll.dll")
↓
拿到 ntdll.dll 基址
↓
GetProcAddress(hNtdll, "NtQueryInformationProcess")
↓
拿到 NtQueryInformationProcess 函数地址
↓
NtQuery(hProcess, ProcessWow64Information, ...)
↓
得到结果：0=64 位，非 0=32 位 (WOW64)
```

● NtQueryInformationProcess 详解

这是 ntdll.dll 提供的一个 NativeAPI（原生 API），比 OpenProcess、EnumProcessModules 更底层。

函数原型：

```
NTSTATUS NtQueryInformationProcess(
    HANDLE ProcessHandle,           //目标进程句柄
    PROCESSINFOCLASS ProcessInformationClass, //要问什么
    PVOID ProcessInformation,       //系统把答案写到这里
    ULONG ProcessInformationLength,  //缓冲区大小
    PULONG ReturnLength             //实际写入的字节数（可传 NULL）
);
```

- 第一个参数：OpenProcess 拿到的句柄。
- 第二个参数：本节课只用 ProcessWow64Information。
- 第三个参数：传一个 ULONG_PTR 变量的地址，系统会往里面填结果。
- 第四、五个参数：按固定值传（sizeof(ULONG_PTR)和 NULL）。

ProcessWow64Information 的返回结果：

值	含义
0	目标进程是纯 64 位。
非 0	目标进程是 32 位 (WOW64)，返回的数值是该进程的 64 位 PEB 地址。

物理意义：问系统“这进程是不是 WOW64？”系统回答“不是 (0)”或“是的，它的 64 位 PEB 在 0x7FF...”。

● 32/64 位 PEB 偏移差异对照表

如果确认目标是 32 位 WOW64 进程，那么必须用下面这套偏移量去读它的 PEB：

目标内容	64 位 PEB 偏移	32 位 PEB 偏移
Ldr	PEB+0x18	PEB+0x0C
InMemoryOrderModuleList	Ldr+0x20	Ldr+0x14

LDR_DATA_TABLE_ENTRY 的 InMemoryOrderLinks	+0x10	+0x08
DllBase	+0x30	+0x18
BaseDllName (UNICODE_STRING)	+0x58	+0x2C
EntryPoint	+0x38	+0x1C
SizeOfImage	+0x40	+0x20